

# Police Shootings Analysis

A Deep Dive: every dataset, every chart, every line that matters

Explainer for `police-shootings-analysis`

## Read this first: the subject

This project analyses a dataset in which every row is a person who was killed. The documents in this folder, and the code they describe, deal with that data the only way that is defensible: descriptively, precisely, and with the limits stated out loud.

Nothing in this project establishes a cause of anything. Every chart is a count or a correlation over a fixed historical extract. Where a number is small, this document says it is small. Where a chart is mislabelled or misleading, this document says so plainly rather than repeating the label. Several charts in this project are in fact wrong, and Section 10 is where they are named.

## Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Orientation: what this project actually is</b>                  | <b>3</b>  |
| 1.1      | What the project is not . . . . .                                  | 3         |
| <b>2</b> | <b>The subject, and what this data cannot tell you</b>             | <b>3</b>  |
| 2.1      | Provenance . . . . .                                               | 3         |
| 2.2      | The limits, stated plainly . . . . .                               | 4         |
| <b>3</b> | <b>The five datasets</b>                                           | <b>5</b>  |
| 3.1      | The fatality schema . . . . .                                      | 5         |
| 3.2      | Two file-format quirks that the code silently absorbs . . . . .    | 5         |
| 3.3      | One inconsistency in the column names . . . . .                    | 6         |
| <b>4</b> | <b>Repository map</b>                                              | <b>6</b>  |
| <b>5</b> | <b>Architecture</b>                                                | <b>6</b>  |
| <b>6</b> | <b>The static script: <code>analyse_deaths.py</code></b>           | <b>7</b>  |
| 6.1      | Decision 1: the headless backend . . . . .                         | 7         |
| 6.2      | Decision 2: the chart counter . . . . .                            | 8         |
| 6.3      | Decision 3: load everything, then <code>fillna(0)</code> . . . . . | 8         |
| 6.4      | Decision 4: the three-library comparison . . . . .                 | 9         |
| 6.5      | The complete chart inventory . . . . .                             | 10        |
| 6.6      | The racial-makeup aggregation, lines 200–237 . . . . .             | 11        |
| 6.7      | <code>convert_country_code</code> and the second map . . . . .     | 12        |
| <b>7</b> | <b>The dashboard</b>                                               | <b>12</b> |
| 7.1      | <code>data.py</code> : the shared layer . . . . .                  | 12        |
| 7.2      | <code>app.py</code> : three figures and a stat row . . . . .       | 13        |
| 7.3      | The request cycle . . . . .                                        | 14        |
| 7.4      | <code>templates/index.html</code> . . . . .                        | 14        |
| <b>8</b> | <b>Dependencies</b>                                                | <b>16</b> |

---

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| <b>9</b>  | <b>Verified run</b>                                               | <b>16</b> |
| <b>10</b> | <b>Honest findings</b>                                            | <b>17</b> |
| 10.1      | Charts 04, 05 and 06 plot the wrong quantity entirely . . . . .   | 17        |
| 10.2      | The income dataset is loaded and never used . . . . .             | 18        |
| 10.3      | The <code>fillna(0)</code> ordering . . . . .                     | 18        |
| 10.4      | Chart 12 claims to be stacked and is not . . . . .                | 18        |
| 10.5      | <code>scatter={'alpha': ...}</code> is silently ignored . . . . . | 18        |
| 10.6      | Smaller findings . . . . .                                        | 19        |
| 10.7      | What the project gets right . . . . .                             | 19        |
| <b>11</b> | <b>If this were picked up tomorrow</b>                            | <b>19</b> |

## 1 Orientation: what this project actually is

### The one-sentence version

`police-shootings-analysis` is a guided data-analysis exercise that reads five fixed CSV files, produces 24 static charts from them with a single top-to-bottom Python script, and additionally serves three of those views as an interactive web dashboard that lets you filter the fatality records by state, year, and armed status.

It is worth being blunt about the category this project belongs to, because it changes what counts as a fair criticism. This is not a research instrument, not a data pipeline, and not a live service. It began as a guided exercise from a Udemy “100 Days of Code” course, and its original purpose was to practise three things: joining several real-world datasets, cleaning inconsistent data, and drawing the same chart in three different plotting libraries in order to compare their interfaces. The interactive dashboard is later work built on top of the same data.

So the project has two entry points that share one subject and almost no code:

- `analyse_deaths.py` is a one-shot script. You run it, it writes 24 chart files into `output/`, and it exits. It is the original exercise, and it is the only thing that touches the four Census-derived datasets.
- `app.py`, supported by `data.py` and `templates/index.html`, is a Flask web dashboard. You run it, it serves a page on your own machine, and the page lets you filter the fatality data and watch three charts recompute. It touches only the fatality dataset.

### Jargon: Entry point

The file you actually run to start a program. A project with two entry points has two different ways in, which do two different jobs. Here, running `python analyse_deaths.py` and running `python app.py` are unrelated activities that happen to read the same CSV file from disk.

### Jargon: CSV

Comma-Separated Values: a plain-text table where each line is a row and commas separate the columns. The oldest and most universal data-exchange format there is. It carries no type information at all, which is the source of a good share of this project’s difficulties: a column that looks like numbers is, as far as the file is concerned, just text.

### 1.1 What the project is not

Neither entry point ingests anything new. Both read the same frozen snapshot of CSV files that ship in the repository. There is no scheduled job, no database, no API client, and no mechanism anywhere in the code to fetch an updated copy of the source data. A chart titled “Killings by Year” is reporting on 2015 to part of 2017 and nothing since, and the code has no way of knowing that its data is now years old.

## 2 The subject, and what this data cannot tell you

### 2.1 Provenance

The fatality data is `Deaths_by_Police_US.csv`: a fixed extract from The Washington Post’s ongoing database of fatal police shootings in the United States. The copy in this repository holds **2,535 records**, spanning 2015 through part of 2017. The Post’s actual database is still maintained and

is far larger today. This file is a photograph of it taken at one moment, and it is the only version this project will ever see.

The four other CSVs are Census-derived city-level snapshots, mostly dated around 2015: median household income, percentage of over-25s who completed high school, percentage of people below the poverty level, and share of race by city.

## 2.2 The limits, stated plainly

These are not disclaimers bolted on for politeness. They are properties of the data that change how every chart in this project must be read.

### Six things this data cannot support

1. **No causation, anywhere.** Every relationship in this project is a count or a correlation. The strongest statistical relationship the code computes is the association between a city’s poverty rate and its high school completion rate, which this document verified at a Pearson correlation of about  $-0.50$ . That is a real association in the joined data and it is not evidence that either one causes the other.
2. **These are recorded incidents, not incidents.** Every row exists because the Post’s process captured it. Counts are counts of records. Any variation in what gets reported, and by whom, is invisible here and indistinguishable from variation in what happened.
3. **Raw counts are not rates.** The choropleth map and the “top 10 cities” chart rank by absolute number of records, with no population denominator. California leads with 424 records and California is also by a wide margin the most populous state. The chart in `analyse_deaths.py` asks in a code comment “Which cities are the most dangerous?”. The chart does not answer that question and cannot: it ranks by count, so it substantially ranks by population.
4. **Some cross-tabulations are tiny.** The race-by-top-10-city chart is built from the ten most common (city, race) pairs in the whole dataset. This document verified that the largest of those ten pairs holds 21 records and the smallest holds 10. Individual bars represent a handful of people. No stable conclusion survives at that sample size.
5. **The Census joins are at city level and the fatality data is never joined to them.** Despite the framing, no chart in this project actually cross-references a person’s death against the poverty rate of the place it happened. The Census charts and the fatality charts sit in the same script and are never joined. See Section 10.2.
6. **Missing values were turned into zero, and zero is a value.** The cleaning step replaces every missing cell with the number 0 before anything else happens. That decision propagates into several charts as if it were data. See Section 6.3, which is the single most consequential thing to understand about this codebase.

### Jargon: Correlation

A single number between  $-1$  and  $+1$  describing how tightly two quantities move together. Near  $+1$ : when one rises the other rises. Near  $-1$ : when one rises the other falls. Near  $0$ : no straight-line relationship. It says nothing whatsoever about why, and a correlation is perfectly capable of arising from a third factor driving both.

### 3 The five datasets

| File                                  | Rows   | Cols | Used by            |
|---------------------------------------|--------|------|--------------------|
| Deaths_by_Police_US.csv               | 2,535  | 14   | both entry points  |
| Pct_People_Below_Poverty_Level.csv    | 29,329 | 3    | analyse_deaths.py  |
| Pct_Over_25_Completed_High_School.csv | 29,329 | 3    | analyse_deaths.py  |
| Share_of_Race_By_City.csv             | 29,268 | 7    | analyse_deaths.py  |
| Median_Household_Income_2015.csv      | 29,322 | 3    | nothing (see 10.2) |

**Table 1:** Row and column counts verified by loading each file with pandas.

#### 3.1 The fatality schema

Fourteen columns:

|      |                 |        |                         |
|------|-----------------|--------|-------------------------|
| id   | manner_of_death | gender | state                   |
| name | armed           | race   | signs_of_mental_illness |
| date | age             | city   | threat_level            |
|      |                 | flee   | body_camera             |

Two of these deserve care. **race** is a single-letter code (W, B, H, A, N, O) assigned by the Post’s compilers, not self-reported by the person; it is a classification made by a third party after the fact. And **signs\_of\_mental\_illness** is a boolean flag meaning that the reporting captured signs, which is a materially weaker and different claim than a diagnosis. The code comment in `analyse_deaths.py` asks “What percentage of people killed by police have been *diagnosed* with a mental illness?” The column does not record diagnoses and the resulting chart does not answer that question.

#### 3.2 Two file-format quirks that the code silently absorbs

##### Encoding and line endings

None of the five files is UTF-8, the modern default. All five are read with `encoding="windows-1252"`, and that argument is load-bearing: remove it and the load fails or mangles characters. This is correct in the code but is the kind of thing that looks arbitrary until it bites.

Stranger, and undocumented anywhere in the project: three of the five files use a bare carriage return as their line ending rather than a line feed. Verified by byte count: `Median_Household_Income_2015.csv` contains 29,322 carriage returns and *zero* line feeds. `Pct_People_Below_Poverty_Level.csv` and `Share_of_Race_By_City.csv` are the same. The other two use ordinary line feeds. pandas handles this without comment, so the project has never had to notice. It is worth knowing, because a naive `head` or `wc -l` on those files returns nonsense.

##### Jargon: Character encoding

The mapping from the bytes in a file to the characters you see. UTF-8 is the modern universal one. Windows-1252 is an older Microsoft encoding. Read a Windows-1252 file as if it were UTF-8 and any byte outside plain English will either come out as garbage or stop the program.

### 3.3 One inconsistency in the column names

Four of the five files spell the state column `Geographic Area`, with a capital A. The race-share file spells it `Geographic area`, lowercase. This is real, and the code has to work around it, which it does at line 217 with an explicit rename:

```
1 .reset_index().rename(columns={'Geographic area': 'Geographic Area'})
```

## 4 Repository map

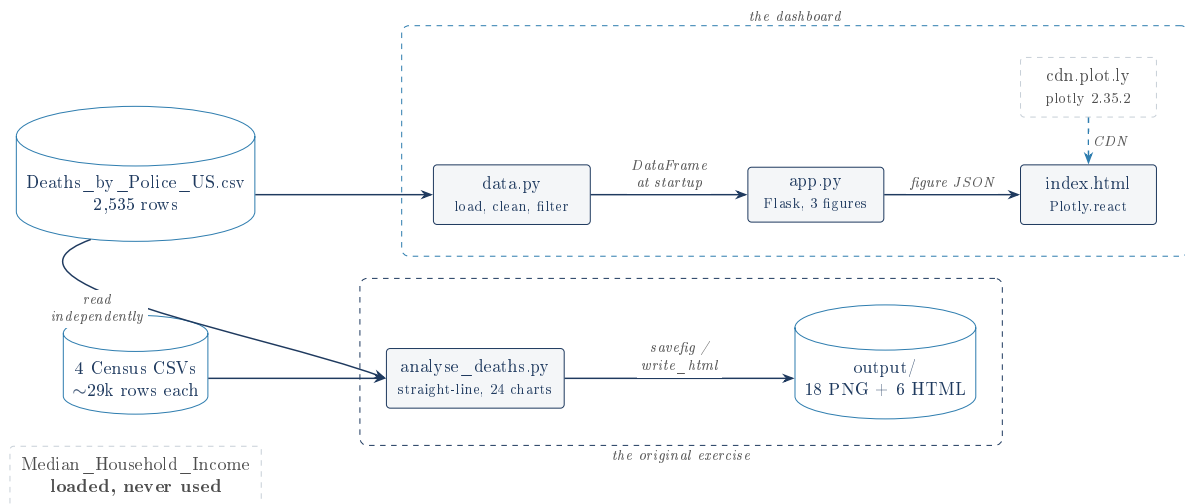
```
police-shootings-analysis
├── analyse_deaths.py  the 392-line static script
├── app.py             Flask dashboard
├── data.py            shared load/clean/filter
├── templates/
│   └── index.html    the whole front end
├── output/           18 committed PNGs
├── Deaths_by_Police_US.csv
├── Median_Household_Income_2015.csv  unused
├── Pct_Over_25_Completed_High_School.csv
├── Pct_People_Below_Poverty_Level.csv
├── Share_of_Race_By_City.csv
├── requirements.txt
├── README.md
└── LICENSE
```

**Figure 1:** Every tracked file. There is no package, no test directory, and no module hierarchy: the three Python files sit flat at the root.

Note what is absent: there are no tests. Not a reduced test suite, none at all. For a project whose entire output is numbers, that is the structural weakness from which most of Section 10 follows. Nothing would have caught a chart that plots the wrong quantity, because nothing checks any chart against anything.

## 5 Architecture

The shape is unusual and worth seeing before reading any code: two entry points that share a data *file* but almost no data *code*.



**Figure 2:** The two halves. They share `Deaths_by_Police_US.csv` on disk and nothing else. `analyse_deaths.py` does not import `data.py`; it reloads and re-cleans the same file with its own inline copy of the same four steps.

### The shared-cleaning claim is half true

`data.py`'s own docstring says it exists "so `app.py` does not duplicate that logic and always applies the exact same cleaning steps the static script does". The README goes further: "This is the only place that logic lives for the dashboard, so `app.py` can't drift from what the numbers actually are."

Read the second half of that sentence carefully, because it is doing a lot of work. `data.py` is the only place the logic lives *for the dashboard*. It is not the only place the logic lives. `analyse_deaths.py` still has its own inline copy of all four cleaning steps and does not import `data.py`. So the two halves absolutely can drift from each other: a fix to the cleaning in one is invisible to the other. The abstraction was extracted for the new code and the old code was left pointing at a duplicate. The README is technically accurate and rhetorically misleading, and an interviewer who reads `data.py` and then greps for its importers will find this in about thirty seconds.

## 6 The static script: `analyse_deaths.py`

392 lines, no classes, and exactly two small functions. It runs top to bottom and every chart is written inline at module level. Understanding it is mostly a matter of understanding four decisions made in the first fifty lines, after which the remaining 340 lines are variations on "build a chart, save it, close it".

### 6.1 Decision 1: the headless backend

```

1 import matplotlib
2 matplotlib.use('Agg') # headless-safe backend so the script runs without a display
  
```

**Listing 1:** Lines 1–2. The first two lines of the file, and the order matters.

#### Jargon: Backend (matplotlib)

The thing matplotlib draws *onto*. The default backend tries to open a window on your screen. On a server, or in a terminal with no graphical display, there is no screen to open a window on. `Agg` is a backend that draws into an in-memory image instead, which can then be written

to a file. Selecting it is what makes this script runnable anywhere.

`matplotlib.use('Agg')` must be called before `matplotlib.pyplot` is imported, which is precisely why these two lines sit above every other import in the file, in violation of the usual convention of grouping imports together. That is not sloppiness; it is required, and it is the correct fix. The original exercise was written for a Jupyter notebook and called `plt.show()` after every chart, which in a plain script silently does nothing.

## 6.2 Decision 2: the chart counter

```
1 base_path = os.path.abspath(os.path.dirname(sys.argv[0]))
2 output_dir = os.path.join(base_path, 'output')
3 os.makedirs(output_dir, exist_ok=True)
4 _chart_index = 0
5
6
7 def next_chart_path(extension):
8     global _chart_index
9     _chart_index += 1
10    return os.path.join(output_dir, f'chart_{_chart_index:02d}.{extension}')
```

**Listing 2:** Lines 26–35. Path setup and the only stateful helper in the file.

Every chart is saved to `next_chart_path('png')` or `next_chart_path('html')`. The counter is shared across both formats, which is why the committed `output/` folder has gaps in it: `chart_03`, `chart_06`, `chart_08`, `chart_15`, `chart_22` and `chart_23` are the six HTML files and are not committed, so the 18 committed PNGs are numbered 01, 02, 04, 05, 07, 09–14, 16–21 and 24.

### The counter makes the charts positionally addressed

Chart numbering is derived purely from execution order. Insert one new chart at line 100 and every file after it is renumbered, so the PNG a reader thought was “the armed-status chart” silently becomes something else. Nothing outside the script records what `chart_16.png` is supposed to be. The 18 committed PNGs are therefore only meaningful in conjunction with the exact version of the script that produced them, and nothing enforces that pairing. The `base_path` derivation is also subtly better than it looks: `os.path.dirname(sys.argv[0])` resolves relative to the *script*, not the current working directory, so the script finds its CSVs no matter where you invoke it from. That is the right call and it was made deliberately.

## 6.3 Decision 3: load everything, then `fillna(0)`

```
1 # Replace 0 with NaN in all columns and rows in all dataframes
2 df_hh_income = df_hh_income.fillna(0)
3 df_pct_poverty = df_pct_poverty.fillna(0)
4 df_pct_completed_hs = df_pct_completed_hs.fillna(0)
5 df_share_race_city = df_share_race_city.fillna(0)
6 df_fatalities = df_fatalities.fillna(0)
```

**Listing 3:** Lines 45–50. The most consequential six lines in the project.

### Jargon: NaN

“Not a Number”: the special floating-point value pandas uses to mean *missing*. It is deliberately contagious. The average of a column containing NaN is NaN unless you explicitly say what to do about it, which is a feature: it makes you decide, rather than quietly producing a wrong number.



### The comment says the exact opposite of the code

The comment reads “Replace 0 with NaN”. The code replaces NaN with 0. It is not a loose paraphrase; it is backwards, and it is backwards in the direction that matters, because those two operations have opposite consequences. Replacing 0 with NaN would make missing data *propagate* and force a decision. Replacing NaN with 0 makes missing data *disappear* into a plausible-looking number.

This one is worth admitting cheerfully in an interview: it is a leftover comment from an exercise brief, and it is a good illustration of why a comment that is not tested is a liability.

And the consequences are real, not theoretical. Because `fillna(0)` runs *before* any of the `pd.to_numeric` coercion calls, every missing value has already become the number 0 by the time the `errors='coerce'` that was supposed to turn bad values into NaN gets a chance to run. This document verified the following on the actual data:

| Column | Missing became | Count    | Where it surfaces                                       |
|--------|----------------|----------|---------------------------------------------------------|
| age    | the number 0   | 77 rows  | chart 17, the age histogram and KDE                     |
| race   | the number 0   | 195 rows | chart 18, as a bar literally labelled 0                 |
| armed  | the number 0   | present  | filtered out in <code>app.py</code> , not in the script |

**Table 2:** Verified by loading the CSV and applying the script’s own cleaning steps.

The age case is the clearest harm. Chart 17 is a histogram and kernel density estimate of the ages of people killed. 77 of the 2,535 records had no recorded age, and all 77 are now plotted as though the person was zero years old. Roughly 3% of the distribution’s mass sits at the extreme left edge, dragging the density curve toward it, and there is no visual cue that those points are fabricated. A reader would take them for infants.

### The correct order

Coerce first, fill second, and only if you have a defensible reason to fill at all: `pd.to_numeric(col, errors='coerce')` to turn genuine junk into NaN, and then leave NaN as NaN so that pandas excludes it from means and plots. Interestingly, the dashboard’s `data.py` inherited the same inverted order, so both halves of the project share this bug. Worth noting for a defence: this is one bug, made once, in the original exercise, and copied.

Note that `app.py` partly compensates for its own inherited version of this: at line 125 it writes `filtered_df[filtered_df['armed'] != 0]` to exclude the filled-in zeros from the “most common armed status” statistic, and `data.py`’s `filter_options` similarly strips 0 from the dropdown lists. So the author did notice the problem in the dashboard and patched it at the point of use, rather than at the source. The static script never got the same treatment.

## 6.4 Decision 4: the three-library comparison

The pedagogical core of the exercise. The first six charts are three questions each answered twice or three times, in matplotlib, seaborn, and plotly:

| Chart | Question                               | Library        | Format |
|-------|----------------------------------------|----------------|--------|
| 01    | Poverty rate by state                  | matplotlib     | PNG    |
| 02    | Poverty rate by state                  | seaborn        | PNG    |
| 03    | Poverty rate by state                  | plotly.express | HTML   |
| 04    | “High school graduation rate” by state | matplotlib     | PNG    |
| 05    | “High school graduation rate” by state | seaborn        | PNG    |
| 06    | “High school graduation rate” by state | plotly.express | HTML   |

**Table 3:** The quotation marks around charts 04–06 are load-bearing. See Section 10.1.

#### Jargon: matplotlib, seaborn, plotly

Three Python plotting libraries at three levels. matplotlib is the low-level foundation: you place the bars yourself. seaborn is built on matplotlib and is statistics-aware: you hand it a dataframe and a column name and it aggregates for you. plotly produces interactive charts that render in a browser with hover and zoom, as HTML rather than a flat image.

That seaborn “aggregates for you” is not a throwaway. It matters directly at chart 07, where `sns.lineplot(x='Geographic Area', y='poverty_rate', data=df_pct_poverty)` is handed 29,329 city rows and silently computes the mean per state, with a bootstrapped confidence interval, and draws that. The author gets a correct state-level average without writing an aggregation. matplotlib at chart 01, handed the same 29,329 rows via `plt.bar(x=..., height=...)`, does no such thing: it draws 29,329 overlapping bars onto 50-odd x positions. The two charts claim the same title and are computed completely differently. That is genuinely the most interesting thing the three-library comparison reveals, and the project does not remark on it.

## 6.5 The complete chart inventory

| #  | Chart                         | Built with        | Status                           |
|----|-------------------------------|-------------------|----------------------------------|
| 01 | Poverty rate by state (bar)   | matplotlib        | unaggregated, see above          |
| 02 | Poverty rate by state (bar)   | seaborn           | mean per state                   |
| 03 | Poverty rate by state (bar)   | plotly            | HTML, not committed              |
| 04 | “HS graduation rate” by state | matplotlib        | <b>plots city counts</b>         |
| 05 | “HS graduation rate” by state | seaborn           | <b>plots city counts</b>         |
| 06 | “HS graduation rate” by state | plotly            | <b>plots city counts</b>         |
| 07 | Poverty vs HS rate (line)     | seaborn           | correct, means per state         |
| 08 | Poverty vs HS rate (line)     | plotly            | HTML, not committed              |
| 09 | Poverty vs HS (joint KDE)     | seaborn           | needs scipy                      |
| 10 | Poverty vs HS (lmplot)        | seaborn           | alpha silently ignored           |
| 11 | Poverty vs HS (regplot)       | seaborn           | alpha silently ignored           |
| 12 | “Percentage by Ethnicity”     | matplotlib        | <b>overlapping, not stacked</b>  |
| 13 | Gender split (pie)            | pandas/matplotlib | 2,428 M / 107 F                  |
| 14 | Gender split (bar)            | pandas/matplotlib | same data as 13                  |
| 15 | Age by manner of death (box)  | plotly            | HTML, not committed              |
| 16 | Armed status, top 5 (bar)     | pandas/matplotlib |                                  |
| 17 | Age distribution (hist + KDE) | pandas/matplotlib | <b>77 fake zeros</b>             |
| 18 | Race (bar)                    | pandas/matplotlib | <b>195 rows in a bar named 0</b> |
| 19 | Signs of mental illness (pie) | pandas/matplotlib | 633 of 2,535                     |
| 20 | Top 10 cities (bar)           | pandas/matplotlib | counts, not rates                |
| 21 | Top 10 (city, race) pairs     | seaborn           | <b>n between 10 and 21</b>       |
| 22 | Choropleth, USA scope         | plotly            | HTML, not committed              |
| 23 | Choropleth, world scope       | plotly            | <b>meaningless, see 6.7</b>      |
| 24 | Killings per year (bar)       | pandas/matplotlib | 991 / 963 / 581                  |

**Table 4:** All 24 charts. Verified by running the script: it produced exactly these 24 files.

## 6.6 The racial-makeup aggregation, lines 200–237

This is the most intricate piece of pandas in the file and it is worth walking through, because the arithmetic is right and the drawing is wrong.

```
1 count_df_share_race_city = df_share_race_city['Geographic area'].value_counts()
2 white_percent = df_share_race_city.groupby('Geographic area')['share_white'].sum() /
    count_df_share_race_city
3 # ... same for black, native_american, asian, hispanic ...
4 white_percent = white_percent.rename('White')
5 # ...
6 merged_series = pd.concat(
7     [white_percent, black_percent, native_american_percent, asian_percent, hispanic_percent],
8     axis=1
9 ).reset_index().rename(columns={'Geographic area': 'Geographic Area'})
```

**Listing 4:** Lines 200–217, condensed. Compute a per-state mean the manual way.

Sum divided by count is a mean, computed the long way round. `.groupby(...).mean()` would do it in one call. The result is an *unweighted mean across cities*: a village of 80 people counts exactly as much as a city of 800,000 in its state’s figure. That is a real methodological choice and it is nowhere acknowledged. It is defensible as “the average city in this state looks like this”; it is not the racial makeup of the state, which is what the chart’s axis label implies.

### Jargon: groupby

Split a table into groups sharing a key (here, the state), compute something per group, and reassemble the answers into one row per group. The single most important operation in dataframe work.

Then it is drawn:

```
1 column_names = ['Geographic Area', 'White', 'Black', 'Native American', 'Asian', 'Hispanic']
2 plt.figure(figsize=(9, 7), dpi=130)
3 x = merged_series['Geographic Area']
4
5 for col in column_names[1:]:
6     y = merged_series[col]
7     plt.bar(x, y, label=col, alpha=0.7)
```

**Listing 5:** Lines 220–233. Five bar calls, all starting at zero.

### Chart 12 is not a stacked bar chart, and the README says it is

A stacked bar chart requires each segment to start where the previous one ended, which in matplotlib means passing `bottom=` the running total. This loop never passes `bottom`. All five bars are drawn from zero, on top of each other, at 70% opacity.

The rendered chart confirms it: this document opened `output/chart_12.png` and the White bar, being the tallest in almost every state, simply covers the four drawn after it wherever they are shorter. The reader sees the White share and muddy translucent slivers. The chart is close to unreadable, and where it is readable it is being read wrong: a viewer familiar with stacked bars will read a bar reaching 92 as “92% cumulative” when it means “the White share is 92”.

The README describes this as a “stacked bar chart”. It is not one. The fix is one keyword argument: track a running `bottom` across loop iterations.

The cosmetic tell that something is off: `column_names` carries ‘Geographic Area’ at index 0 purely so it can be sliced off again with `[1:]` on the next line.

## 6.7 convert\_country\_code and the second map

```

1 def convert_country_code(code):
2     try:
3         country = pycountry.countries.get(alpha_2=code)
4         if country is None:
5             return code
6         return country.alpha_3
7     except (ValueError, KeyError):
8         return code

```

**Listing 6:** Lines 13–21. The only other function in the file.

This converts a two-letter ISO country code to a three-letter one. It is applied to US state postal abbreviations, which are not ISO country codes, so `pycountry` returns `None` for essentially all of them and the function returns its input unchanged. The `if country is None` guard exists because without it the whole script crashed with `AttributeError: 'NoneType' object has no attribute 'alpha_3'`.

### The guard fixed the crash and preserved the confusion

The function is now a well-guarded no-op. It is a leftover from a template aimed at country-level choropleths, and `pycountry` is a dependency in `requirements.txt` solely to support it. Worse, some state codes *are* valid ISO country codes, and for those the function does not no-op. It silently rewrites them. This document has not enumerated which of the 51 values collide, so treat that as **inferred rather than verified**: the mechanism is certain from reading `pycountry`'s contract, the specific blast radius on this dataset was not measured. It does not affect chart 22, which passes `locationmode='USA-states'` and would ignore a three-letter code anyway. It is chart 23, the world-scope map, that is plotting a mixture of state postal codes and accidentally-converted country codes onto a map of the world. The README concedes chart 23 is “mostly illustrative”. It is not illustrative of anything; it should be deleted along with the function and the dependency.

## 7 The dashboard

Three files, roughly 230 lines of Python and 250 of HTML. It is markedly better engineered than the script it sits beside.

### 7.1 data.py: the shared layer

Sixty-three lines, three functions, no side effects beyond reading a file.

```

1 def load_fatalities():
2     """Load and clean Deaths_by_Police_US.csv, same steps as analyse_deaths.py."""
3     df = pd.read_csv(os.path.join(BASE_PATH, 'Deaths_by_Police_US.csv'), encoding='windows
4                       -1252')
5     df = df.fillna(0)
6     df['age'] = pd.to_numeric(df['age'], errors='coerce')
7     df['date'] = pd.to_datetime(df['date'], format='mixed', errors='coerce')
8     df['year'] = df['date'].dt.year
9     return df

```

**Listing 7:** `data.py` lines 29–36. The whole cleaning path.

`format='mixed'` is the interesting one. The `date` column is mostly `M/D/YYYY` but some rows are `DD/MM/YY`, so a plain `pd.to_datetime` raises `ValueError: time data "13/01/15" doesn't match format`. `format='mixed'` infers per value. This document verified that it succeeds completely on

this file: **zero** rows become NaT. The `errors='coerce'` safety net is never actually needed, but it is correctly there.

#### Jargon: NaT

“Not a Time”: the NaN equivalent for dates. What `errors='coerce'` produces for a value it cannot parse.

`apply_filters` is worth reading for its one subtlety:

```
1 def apply_filters(df, state=None, year=None, armed=None):
2     out = df
3     if state and state != 'all':
4         out = out[out['state'] == state]
5     if year and year != 'all':
6         out = out[out['year'] == int(year)]
7     if armed and armed != 'all':
8         out = out[out['armed'] == armed]
9     return out
```

**Listing 8:** `data.py` lines 52–62.

Each filter is optional and they compose. `out = df` does not copy: it binds a second name to the same object, and each subsequent `out[...]` produces a new filtered view. If no filter applies, the function returns the original dataframe unchanged, which is correct only because every caller treats it as read-only. Nothing enforces that. A single `filtered['x'] = ...` anywhere downstream would mutate the process-wide DF for every future request. Not a bug today, but a sharp edge one line away.

## 7.2 `app.py`: three figures and a stat row

```
1 DF = load_fatalities()
2 STATES, YEARS, ARMED_TYPES = filter_options(DF)
3
4
5 def fig_to_json(fig):
6     return json.loads(fig.to_json())
```

**Listing 9:** `app.py` lines 39–47. Load once, serve many.

Loading at import time rather than per request is the right call for a read-only 2,535-row snapshot: it costs a few hundred milliseconds once instead of on every click.

`fig_to_json` is small and load-bearing. A Plotly figure’s internals are full of numpy `int64` values, which Flask’s JSON encoder does not know how to serialize. Rather than hand `fig.to_dict()` to `jsonify` and discover this at request time, every figure goes through Plotly’s own `to_json`, which handles numpy correctly, and back through `json.loads`. It is a round trip through a string purely to borrow someone else’s encoder. Mildly wasteful, entirely correct, and a genuinely good instinct.

#### Jargon: Serialization

Turning an in-memory object into a stream of bytes that can be sent over a network and rebuilt at the other end. JSON is the usual choice for web work. The catch is that JSON only knows a few types, so anything exotic, such as a numpy 64-bit integer, needs an explicit rule.

The three figure builders are near-identical in shape: filter, count, build a `go.Figure`, set a layout. `build_choropleth` is the only one with real logic in it, and it is a nice touch:

```
1 marker_line_color=(
2     [('f97316' if s == selected_state else 'white') for s in state_counts['state']]
```

```

3 ),
4 marker_line_width=(
5     [3 if s == selected_state else 0.5 for s in state_counts['state']]
6 ),

```

**Listing 10:** `app.py` lines 61–66. Per-state border styling.

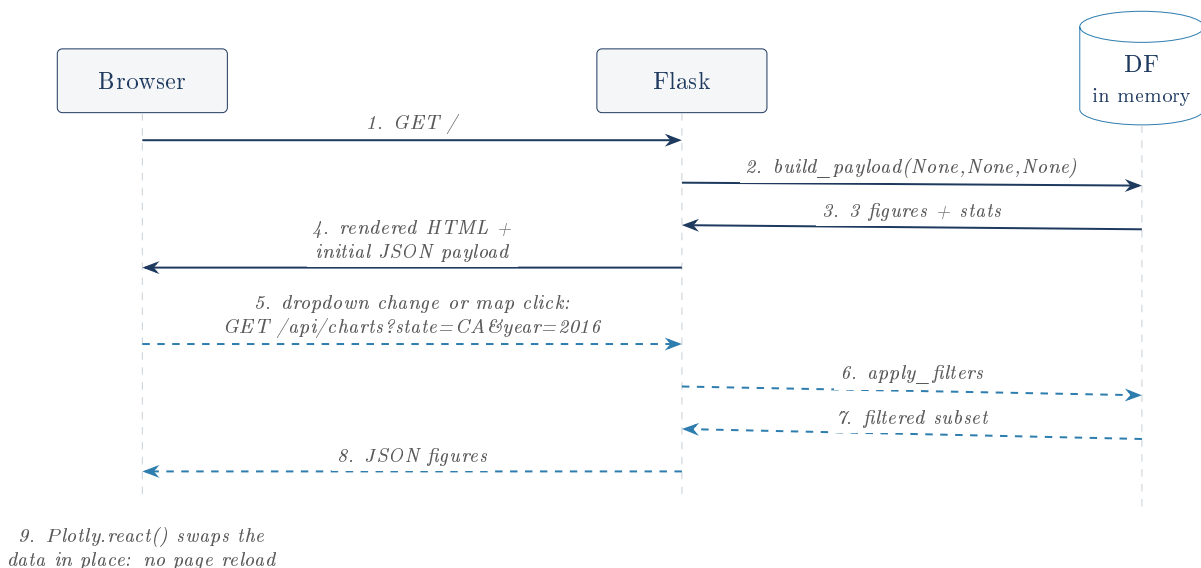
The selected state gets a thick orange border. Passing a *list* rather than a scalar gives per-element styling, which is how you highlight one region of a choropleth without drawing a second trace over it.

#### build\_stats is defensive in the right places

`build_stats` checks if `total == 0` first and returns a payload of Nones rather than letting `value_counts().index[0]` raise `IndexError` on an empty selection. This document tested exactly that path: `?state=WY&year=2017&armed=vehicle` returns `{'total': 0, 'pct_mental_illness': None, 'top_state': None, 'top_armed': None}` and the page renders “n/a”. It works. Whoever wrote it thought about the empty case, which is more than most dashboard code does.

The comparison `filtered_df['signs_of_mental_illness'] == True` carries a `# noqa: E712` to silence the linter that would tell you to write `is True`. Here the linter is wrong and the author is right: on a pandas Series, `== True` is a vectorised element-wise comparison and `is True` would be an identity check on the whole Series object. Suppressing the warning is correct, and knowing why is a good interview answer.

### 7.3 The request cycle



**Figure 3:** First load is server-rendered (steps 1–4) so the page arrives with charts already in it. Every subsequent interaction is steps 5–9 only.

### 7.4 `templates/index.html`

One file: markup, CSS and JavaScript together, no build step, no framework. For a page this size that is the right call.

```

1 const initialPayload = {{ initial_payload_json | safe }};

```

**Listing 11:** Line 182. The server-rendered handoff.

**Jargon: Jinja2 and the safe filter**

Jinja2 is Flask’s template language: `{{ ... }}` is a hole that a Python value is poured into. By default Jinja escapes what it inserts, turning `<` into `&lt;` to prevent injected markup from executing. `| safe` disables that. Here it is necessary, because the value is already a JSON string and escaping it would corrupt it into unparseable text.

**| safe is correct here but the reasoning must be understood**

`| safe` says “I promise this cannot inject markup”. The promise holds because the string comes from `json.dumps` over data the server built itself, not from user input. But it is worth tracing why, because the argument is closer than it looks. `city` and `state` values from the CSV are interpolated into figure JSON, and they reach the page through this unescaped hole. If the CSV contained a city named `</script><script>...`, `json.dumps` would escape the quotes but *not* the forward slash in `</script>`, and the browser’s HTML parser terminates the script block on that sequence regardless of JSON quoting. The data is a trusted, inspected, static CSV from The Washington Post, so this is safe today. It is safe because of a property of the data, not a property of the code. Point a future version at a user-supplied CSV and this line is an XSS vector. The one-word fix is `json.dumps(...).replace('/', '\u002f')`, or moving the payload out of the script tag entirely.

The rest of the JavaScript is 60 lines and does exactly what it needs to:

```
1 document.getElementById('choropleth-plot').on('plotly_click', (evt) => {
2   const point = evt.points && evt.points[0];
3   if (!point) return;
4   const clicked = point.location;
5   if (stateSelect.value === clicked) {
6     stateSelect.value = 'all';
7   } else {
8     stateSelect.value = clicked;
9   }
10  fetchAndRender();
11 });
```

**Listing 12:** Lines 236–246 of `index.html`, JavaScript. Map click sets the filter.

Clicking the already-selected state deselects it. That is a small, thoughtful interaction detail: the map click and the dropdown are two controls over one piece of state, and the click routes *through* the dropdown rather than around it, so they cannot disagree.

`Plotly.react` rather than `Plotly.newPlot` is the other good call: it diffs against the existing chart and updates in place, preserving the user’s zoom and pan, instead of tearing the chart down and rebuilding it.

**Three real weaknesses in the front end**

1. **The CDN is an unpinned single point of failure.** Line 6 loads Plotly from `cdn.plot.ly`. The dashboard therefore requires internet access to render anything, despite being a local app over local data, and there is no Subresource Integrity hash, so the page executes whatever that host returns.
2. **GET / ignores its query string.** `index()` hardcodes `build_payload(state=None, year=None, armed=None)`. Verified: requesting `/?state=CA` renders the full 2,535-record view. So the filter state is never in the URL, and a filtered view cannot be bookmarked, shared, or restored by reload. The `/api/charts` endpoint parses exactly these parameters correctly, so the plumbing already exists and `index()` simply does not use it.
3. **`fetchAndRender` swallows failures.** The `try/finally` hides the loading indicator but has

no `catch`. If the fetch fails, the promise rejects, the indicator hides, and the page silently keeps showing stale charts that no longer match the dropdowns. The user is shown wrong data with no error.

## 8 Dependencies

| Package    | Pinned  | Role                                                     |
|------------|---------|----------------------------------------------------------|
| pandas     | 3.0.3   | every dataframe operation in the project                 |
| matplotlib | 3.11.0  | the 18 PNGs; Agg backend                                 |
| seaborn    | 0.13.2  | charts 02, 05, 07, 09, 10, 11, 21                        |
| plotly     | 6.8.0   | the 6 HTML charts and all 3 dashboard figures            |
| pycountry  | 26.2.16 | <code>convert_country_code</code> only; effectively dead |
| scipy      | 1.18.0  | <b>never imported</b> ; see below                        |
| Flask      | 3.1.3   | the dashboard                                            |

**Table 5:** `requirements.txt` in full. Every version is pinned exactly, which is right for a reproducible analysis.

### scipy is the interesting one

`scipy` appears nowhere in any `import` statement in this project, and the project does not work without it. Both `df['age'].plot(kind='kde')` (chart 17) and `sns.jointplot(..., kind='kde')` (chart 09) delegate their density estimation to `scipy.stats.gaussian_kde` lazily, at draw time. Without `scipy` installed they fail at runtime with `ModuleNotFoundError`, deep inside a plotting call, with nothing in the source to suggest why.

Listing it explicitly is exactly right. A transitive dependency your code depends on *behaviourally* is a direct dependency, whatever the import graph says.

### Jargon: Transitive dependency

A package you did not ask for, installed because something you did ask for needs it. The trap: if your code relies on one without declaring it, your install works right up until the intermediate package drops it, and then it breaks for reasons that have nothing to do with anything you changed.

### Jargon: KDE (Kernel Density Estimate)

A smooth curve estimating the shape of a distribution. A histogram chops data into bins and counts; a KDE places a small bump over each data point and adds them up, giving a continuous curve. It is a smoothed guess at the underlying shape, not a measurement, and it will happily smooth a cluster of fabricated zeros into what looks like a real feature. Which, in chart 17, is precisely what it does.

## 9 Verified run

Everything in this section was produced by actually running the code, in a clean virtual environment built from `requirements.txt`, not read off the README.



| Claim                   | Result                                                           | Verdict             |
|-------------------------|------------------------------------------------------------------|---------------------|
| Total fatality records  | 2,535                                                            | confirmed           |
| Gender split            | 2,428 M / 107 F                                                  | confirmed           |
| Signs of mental illness | 633 of 2,535 (25.0%)                                             | confirmed           |
| Top 5 armed             | gun 1,398; knife 373; vehicle 177; unarmed 171; undetermined 117 | confirmed           |
| Top 5 states            | CA 424; TX 225; FL 154; AZ 118; OH 79                            | confirmed           |
| Charts produced         | exactly 24 (18 PNG, 6 HTML)                                      | confirmed           |
| HTML chart size         | 4.6 to 5.5 MB each, 31 MB total                                  | confirmed           |
| Rows parsed to NaT      | 0                                                                | confirmed           |
| Dashboard total         | 2,535, “gun (1398)”, “CA (424)”                                  | confirmed           |
| Empty filter            | returns <b>total:</b> 0, page shows n/a                          | confirmed           |
| Killings per year       | <b>991 / 963 / 581</b> , not 991 / 962 / 582                     | <b>README wrong</b> |
| Script runtime          | <b>6 min 54 s</b> , not “under a minute”                         | <b>README wrong</b> |

**Table 6:** The two failures are minor in size and worth knowing about before an interviewer finds them.

The correlation between city poverty rate and city high school completion rate, over all 29,329 joined rows, is  $-0.4997$ . The merge on **Geographic Area** plus **City** produces exactly 29,329 rows from two 29,329-row inputs, so the join is clean and one-to-one on this data and loses nothing.

## 10 Honest findings

Ordered by how much they would hurt if an interviewer found them first.

### 10.1 Charts 04, 05 and 06 plot the wrong quantity entirely

Three charts titled “High School Graduation Rate by State” plot the number of cities per state

```
1 sorted_percent_complete_hs = df_pct_completed_hs.groupby('Geographic Area')['
    percent_completed_hs'].count().sort_values(
2     ascending=True)
```

`.count()` counts *how many rows are in each group*. It never looks at the values in the column at all. `.mean()` is what a rate requires.

So the series being plotted is the number of cities each state has in the CSV. This document rendered `output/chart_04.png` and read it: the y-axis is labelled “High School Graduation Rate” and runs to **1750**. The tallest bars are PA (1,761), TX (1,710) and CA (1,501). The shortest is DC (1), because the District of Columbia contributes exactly one city row. A graduation rate of 1,761 percent, and the chart carries a straight face.

Verified: the correct `.mean()` version gives TX 75.7, MS 78.5, GA 79.0 at the bottom of the ranking, which are plausible percentages.

This is three of the 24 charts, and one of them (04) is committed to the repository as a PNG. It is also the sole reason the code comment’s question “Which state has the lowest high school graduation rate?” gets the answer DC, which is not a graduation fact about DC but an artefact of DC having one row.

The tell was there to be caught: chart 07 in the same script plots the same quantity correctly, via seaborn’s implicit mean, on a y-axis that tops out near 100. Two charts in one document disagree by a factor of 17 and nobody noticed, because nothing in this project ever compares a chart against an expectation. The one-word fix is `.count()` to `.mean()`.

## 10.2 The income dataset is loaded and never used

### Median\_Household\_Income\_2015.csv feeds nothing

The README's first sentence says the fatality data is “cross-referenced against four US Census datasets (poverty rate, high school completion rate, *median household income*, and racial demographics by city) to look for socioeconomic patterns”.

`df_hh_income` appears exactly twice in the entire project. Line 37 reads the file. Line 46 calls `.fillna(0)` on it. It is never read again, by any chart, in any file. Verified by `grep` across every Python and HTML file in the repository.

So the count is three Census datasets, not four. Income is not analysed, not joined, and not plotted. Roughly 709 KB is parsed on every run to produce a variable nobody reads.

The larger version of this finding is worth stating too, because an interviewer will get there: **the fatality data is never joined to any Census data at all**. Not to poverty, not to high school completion, not to race share. The Census charts (01–12) and the fatality charts (13–24) are two unrelated analyses that share a script. “Cross-referenced ... to look for socioeconomic patterns” describes something the code does not do. The honest description is: this project charts some Census data, and separately charts some fatality data, and the socioeconomic cross-reference remains an aspiration.

## 10.3 The `fillna(0)` ordering

Covered in full at Section 6.3. In summary: missing values become 0 before the coercion that was supposed to make them NaN, putting 77 fabricated zero-year-old people into the age distribution and a bar labelled 0 holding 195 records into the race chart. The inline comment describes the opposite operation. Both entry points share the bug because `data.py` copied it.

## 10.4 Chart 12 claims to be stacked and is not

Covered at Section 6.6.

## 10.5 `scatter={'alpha': ...}` is silently ignored

### A keyword that looks right, does nothing, and raises no error

```
1 sns.lmplot(data=merge_poverty_hs, x='poverty_rate', y='percent_completed_hs', scatter={'
    alpha': 0.3},
2           line_kws={'color': 'red'}, height=7, aspect=1.5)
3 # ... and at line 179:
4 sns.regplot(data=merge_poverty_hs, x='poverty_rate', y='percent_completed_hs', scatter={'
    alpha': 0.7},
5            line_kws={'color': 'red'})
```

Verified by inspecting the installed seaborn 0.13.2 signature: `regplot` and `lmplot` both take `scatter`, a **boolean** defaulting to `True`, and both take `scatter_kws`, the dict of styling that was actually intended. A non-empty dict is truthy, so `scatter={'alpha': 0.3}` means `scatter=True` and the `alpha` is discarded without a warning.

The author clearly knew the right pattern, because `line_kws={'color': 'red'}` sitting immediately beside it is exactly correct and does work. So charts 10 and 11 are overplotted at full opacity across 29,329 points, when the intent was transparency to reveal density. The charts are not *wrong*, they are just not what was asked for, and Python never said a word. The fix is `scatter_kws={'alpha': 0.3}`.

## 10.6 Smaller findings

- **An orphaned matplotlib figure.** Line 267 calls `plt.figure(figsize=(9, 7), dpi=130)` immediately before building a *plotly* box plot. The matplotlib figure is never drawn to, never saved and never closed. Harmless, and a clear fossil of copy-paste.
- **Chart 01 draws 29,329 bars onto 50 x-positions.** Discussed above. matplotlib does not aggregate; the resulting picture is not the state poverty rate but every city's rate overplotted. Chart 02, the seaborn version of the same title, does aggregate. The two charts claim to show the same thing and do not.
- **“Which cities are the most dangerous?”** A code comment at line 328 asking a question the chart beneath it cannot answer. Ranking by raw count ranks by population.
- **No tests, no schema validation.** A renamed CSV header would surface as a `KeyError` several hundred lines into a chart block, not as a clear failure at load.
- **The README slightly understates the script.** It says “one 380-line procedural script”; it is 392 lines. Trivial, but the README's numbers are checkable and two of them (year counts, runtime) are already wrong, which makes the rest worth checking.

## 10.7 What the project gets right

The honesty rule cuts both ways, and a list of only faults would misrepresent the code.

- The `Agg` backend and file output make the script genuinely reproducible and runnable anywhere. This was a real fix to a real problem and it is the single most valuable change made to the original exercise.
- `base_path` derived from `sys.argv[0]` means the script works from any working directory.
- Every dependency version is pinned exactly, including the one that is only needed behaviourally.
- `format='mixed'` is precisely the right tool for the date column, and verified to parse all 2,535 rows.
- `fig_to_json` routes around a real Flask/numpy incompatibility before it could bite at request time, rather than after.
- `build_stats` handles the empty-selection case, tested and confirmed.
- The `# noqa: E712` is a case of the author being right and the linter being wrong, knowingly.
- The map-click-through-the-dropdown interaction is a genuinely good piece of single-source-of-truth UI design.
- The README's “Data and framing notes” section states the causation and sample-size limits without being asked to, and the dashboard footer repeats them on the page itself. On a subject like this that matters, and most exercise write-ups do not do it.

## 11 If this were picked up tomorrow

In priority order, and each is small:

1. `.count()` to `.mean()` at line 89. One word. Fixes three charts that are currently false.
2. Move `fillna(0)` after the `to_numeric` coercions, or delete it. Fixes the age distribution and the race chart, in both entry points.
3. Either use `df_hh_income` or delete it, and correct the README's “four Census datasets” and its “cross-referenced” framing to match what the code does.

4. Pass `bottom=` in the chart 12 loop, or say “overlapping” in the README.
5. `scatter=` to `scatter_kws=` in charts 10 and 11.
6. Delete `convert_country_code`, chart 23, and the `pycountry` dependency.
7. Correct the README’s year counts to 991 / 963 / 581 and its runtime claim to approximately seven minutes.
8. Make `index()` read the same query parameters `api_charts` already reads, so a filtered view has a URL.
9. Add a `catch` to `fetchAndRender` so a failed request does not leave stale charts on screen.
10. Give each chart a function and a name, so `chart_16.png` means something independent of execution order, and so a single chart can be re-run without paying the full seven minutes.

#### The one-line summary for a defence

The dashboard half is careful, well-factored code with real thought in it. The static-script half is an exercise that was made to run headlessly and then not looked at again, and it contains three charts that are confidently, visibly false. The gap between the two is the whole story of the project, and the reason for the gap is that neither half has a single test.